

Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya (SCSVMV)

Deemed to be University | Accredited by NAAC with "A" Grade
(Sponsored and run by Sri Kanchi Kamakoti Peetam Charitable Trust)

Department of Computer Science and Engineering

Fundamentals Of Machine Learning Lab

Student Name :G.LOKESH SAI

Register Number : 11259A079

Department : CSE -AI&ML

Semester : 3rd

Section: S-2

Subject: Fundamentals Machine Learning Lab

Course Code :

Academic Year : 2nd year

Faculty Name : Dr. R. SIVARAMA KRISHNAN

Sri Chandrasekharendra Saraswathi
Viswa Mahavidyalaya (SCSVMV)
Deemed to be University | Accredited by NAAC with "A" Grade
(Sponsored and run by Sri Kanchi Kamakoti Peetam Charitable Trust)

Fundamentals Of Machine Learning Lab

Exp No.1 : Data Preprocessing in Given Data Set

Program : Data Preprocessing

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv("data.csv")

# Display first 5 rows
print("Original Dataset:")
print(data.head())

# Check missing values
print("\nMissing Values:")
print(data.isnull().sum())

# Fill missing values (if any)
data.fillna(data.mean(numeric_only=True), inplace=True)

# Select numerical columns
X = data[['Hours_Studied', 'Attendance', 'Assignments', 'Marks']]

# Normalize data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("\nNormalized Dataset:")
print(X_scaled.head())
```

Fundamentals Of Machine Learning Lab

Output :

```
Original Dataset:
   StudentID  Hours_Studied  Attendance  Assignments  Marks Result
0          1              2          75            3     45   Fail
1          2              3          80            4     52   Pass
2          3              4          85            5     60   Pass
3          4              1          65            2     35   Fail
4          5              5          90            5     75   Pass

Missing Values:
StudentID       0
Hours_Studied   0
Attendance      0
Assignments     0
Marks           0
Result          0
dtype: int64

Normalized Dataset:
   Hours_Studied  Attendance  Assignments     Marks
0      -0.958215   -0.697680    -0.912871 -0.965884
1      -0.440261   -0.267013     0.000000 -0.603354
2       0.077693    0.163653     0.912871 -0.189034
3      -1.476169   -1.559013    -1.825742 -1.483784
4       0.595647    0.594320     0.912871  0.587817

Process finished with exit code 0
```

Sri Chandrasekharendra Saraswathi
Viswa Mahavidyalaya (SCSVMV)
Deemed to be University | Accredited by NAAC with "A" Grade
(Sponsored and run by Sri Kanchi Kamakoti Peetam Charitable Trust)

Fundamentals Of Machine Learning Lab

Exp No.2 :Data_Split_Up And Cross Validation

Program 2.1 : Data Splitting

```python
# 2.1 Data Splitting
import pandas as pd
from sklearn.model_selection import train_test_split

# Load dataset
data = pd.read_csv("data.csv")

# Features (Input)
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target (Output)
y = data['Result']

# Split data into 80% Training and 20% Testing
X_train, X_test, y_train, y_test = train_test_split(
*arrays: X, y, test_size=0.2, random_state=42
)

# Display sizes
print("Training Data:")
print(X_train)

print("\nTesting Data:")

print(X_test)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test)
```

Fundamentals Of Machine Learning Lab

Output :

```
Training Data:
    Hours_Studied  Attendance  Assignments
8               3          78            4
5               6          95            5
11              5          92            5
3               1          65            2
18              5          91            5
16              7          99            5
13              2          72            3
2               4          85            5
9               4          88            4
19              6          97            5
4               5          90            5
12              6          96            5
7               7          98            5
10              1          60            2
14              4          84            4
6               2          70            3

Testing Data:
    Hours_Studied  Attendance  Assignments
0               2          75            3
17              1          68            2
15              3          79            4
1               3          80            4

Training Samples: 16
Testing Samples: 4

Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

Program 2.2 : Cross validation.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier

# Load dataset
data = pd.read_csv("data.csv")

# Features and Target
X = data[['Hours_Studied', 'Attendance', 'Assignments']]
y = data['Result']

# Split dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
 *arrays: X, y, test_size=0.2, random_state=42
)

print("Training Data Size:", len(X_train))
print("Testing Data Size:", len(X_test))
# Create KNN Model
model = KNeighborsClassifier(n_neighbors=3)


# Perform 5-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5)

print("\nCross Validation Scores:")
print(scores)

print("\nAverage Accuracy:")
print(scores.mean())
```

Fundamentals Of Machine Learning Lab

Output :

```
Training Data Size: 16
Testing Data Size: 4

Cross Validation Scores:
[0.75 1.   1.   1.   1.  ]

Average Accuracy:
0.95

Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

Exp No.3 : Implement Linear Regression

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv("data.csv")

# Input feature
X = data[['Hours_Studied']]

# Target variable
y = data['Marks']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

# Create Linear Regression model
model = LinearRegression()

# Train the model
model.fit(X_train, y_train)

# Predict values
y_pred = model.predict(X_test)

# Evaluate performance
print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
```

```
# Display predictions
print("\nPredicted Marks:")
print(y_pred)

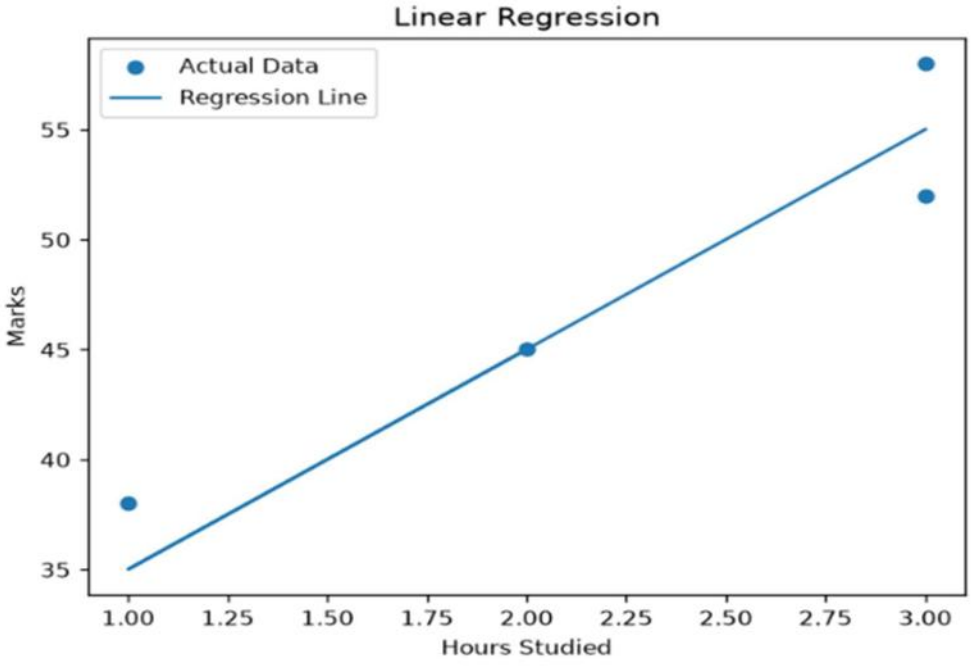
# Graph
plt.scatter(X_test, y_test, label="Actual Data")
plt.plot( *args: X_test, y_pred, label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

Output :

```
Mean Squared Error: 6.750000000000021
R2 Score: 0.879866518353726

Predicted Marks:
[45. 35. 55. 55.]
```

Graph :



Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya (SCSVMV)
Deemed to be University | Accredited by NAAC with "A" Grade
(Sponsored and run by Sri Kanchi Kamakoti Peetam Charitable Trust)

Fundamentals Of Machine Learning Lab

Exp No.4 : Implement Classification Model Using KNN and Compute Evaluation Metrics

Program 4.1 :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
 *arrays: X, y, test_size=0.2, random_state=42
)


# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Fundamentals Of Machine Learning Lab

Output :

```
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Scripts\python.exe
Accuracy: 1.0

Confusion Matrix:
[[2 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2
        Pass       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4


Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

Program 4.2 :

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Fundamentals Of Machine Learning Lab

Output :

```
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Scripts\pyt
Accuracy: 1.0

Confusion Matrix:
[[2 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2
        Pass       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4


Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

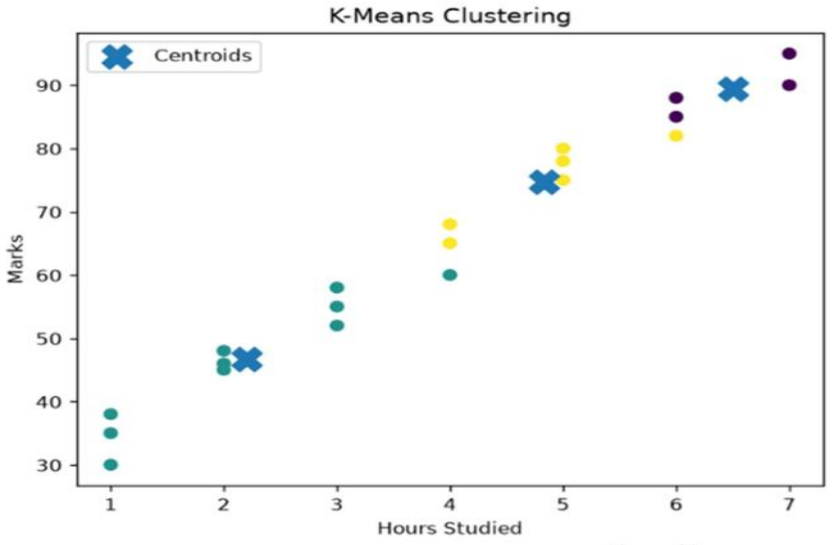
Exp No. 5 : Implement K-Means Clustering

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
# Load dataset
data = pd.read_csv("data.csv")
# Select features
X = data[['Hours_Studied', 'Marks']]
# Create K-Means model
kmeans = KMeans(n_clusters=3, random_state=42)
# Fit the model
kmeans.fit(X)
# Predict cluster labels
clusters = kmeans.labels_
# Add cluster column to dataset
data['Cluster'] = clusters
# Display clustered data
print(data)
# Plot clusters
plt.scatter(X['Hours_Studied'], X['Marks'], c=clusters)
# Plot centroids
plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker='X',
    s=200,
    label='Centroids'
)
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("K-Means Clustering")
plt.legend()
```

Fundamentals Of Machine Learning Lab

Graph :



Output :

```
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\laksh\PycharmProjects\PythonProject1\main.p
    StudentID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              2          75            3     45   Fail        1
1           2              3          80            4     52   Pass        1
2           3              4          85            5     60   Pass        1
3           4              1          65            2     35   Fail        1
4           5              5          90            5     75   Pass        2
5           6              6          95            5     82   Pass        2
6           7              2          70            3     48   Fail        1
7           8              7          98            5     90   Pass        0
8           9              3          78            4     55   Pass        1
9          10              4          88            4     68   Pass        2
10         11              1          60            2     30   Fail        1
11         12              5          92            5     80   Pass        2
12         13              6          96            5     85   Pass        0
13         14              2          72            3     46   Fail        1
14         15              4          84            4     65   Pass        2
15         16              3          79            4     58   Pass        1
16         17              7          99            5     95   Pass        0
17         18              1          68            2     38   Fail        1
18         19              5          91            5     78   Pass        2
19         20              6          97            5     88   Pass        0

Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

Exp No.6 : Implement Hierarchical Clustering

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

# Load dataset
data = pd.read_csv("data.csv")

# Select features
X = data[['Hours_Studied', 'Marks']]

# Create linkage matrix
linked = linkage(X, method='ward')

# Plot Dendrogram
plt.figure(figsize=(8, 5))
dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

# Apply Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=3)

clusters = hc.fit_predict(X)

# Add cluster labels to dataset
data['Cluster'] = clusters

print(data)
```

Fundamentals Of Machine Learning Lab

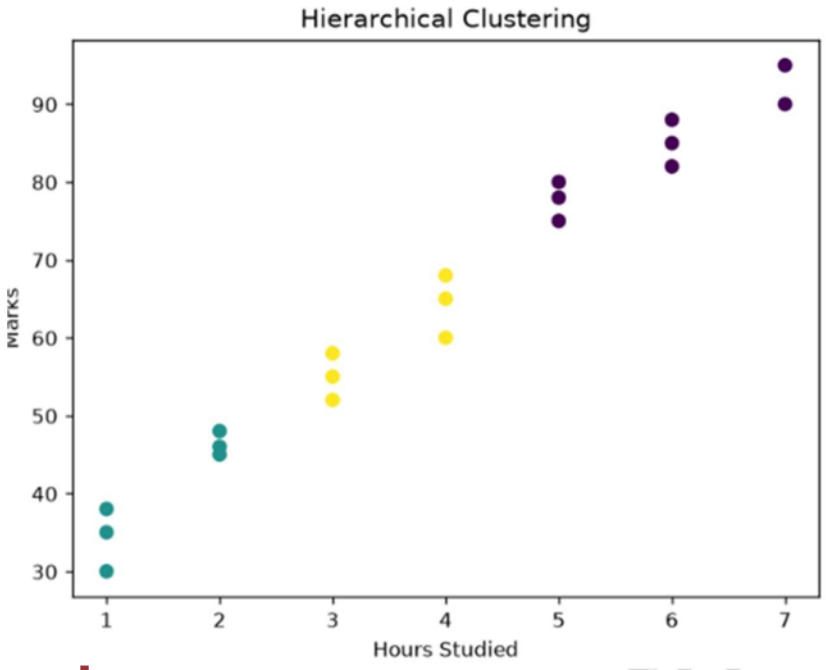
Output :

```
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\laks
    StudentID  Hours_Studied  Attendance  Assignments  Marks Result  Cluster
0           1              2          75            3     45   Fail        1
1           2              3          80            4     52   Pass        2
2           3              4          85            5     60   Pass        2
3           4              1          65            2     35   Fail        1
4           5              5          90            5     75   Pass        0
5           6              6          95            5     82   Pass        0
6           7              2          70            3     48   Fail        1
7           8              7          98            5     90   Pass        0
8           9              3          78            4     55   Pass        2
9          10              4          88            4     68   Pass        2
10         11              1          60            2     30   Fail        1
11         12              5          92            5     80   Pass        0
12         13              6          96            5     85   Pass        0
13         14              2          72            3     46   Fail        1
14         15              4          84            4     65   Pass        2
15         16              3          79            4     58   Pass        2
16         17              7          99            5     95   Pass        0
17         18              1          68            2     38   Fail        1
18         19              5          91            5     78   Pass        0
19         20              6          97            5     88   Pass        0

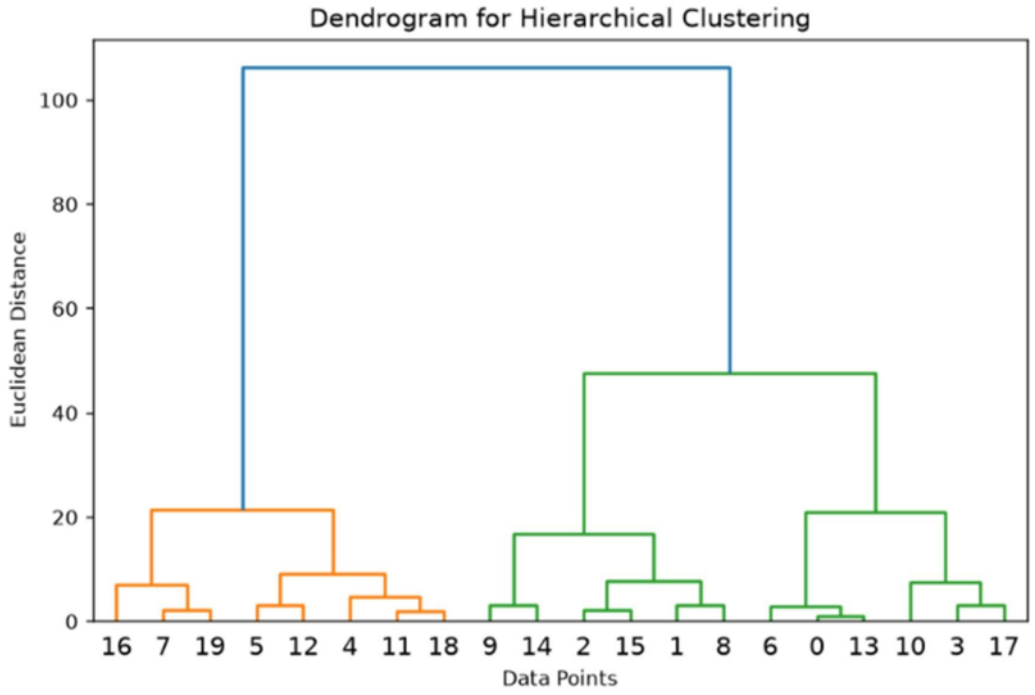
Process finished with exit code 0
```

Fundamentals Of Machine Learning Lab

Graph :



Dendogram :



Fundamentals Of Machine Learning Lab

Exp No.7 : Implement Random Forest and Analyze Performance

Program:

```
# expo no7 implement random forest
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt
data = pd.read_csv("data.csv")
X=data[['Hours_Studied', 'Attendance', 'Assignments']]
y=data['Result']
X_train,X_test,y_train,y_test=train_test_split( *arrays: X,y,test_size=0.2,random_state=42)
model=RandomForestClassifier(n_estimators=100,random_state=42)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
importance=model.feature_importances_
plt.bar(X.columns,importance)
plt.xlabel("Feature Names")
plt.ylabel("Importance")
plt.title("Random Forest Feature Importance")
plt.show()
```

Fundamentals Of Machine Learning Lab

Output :

```
Accuracy: 1.0

Confusion Matrix:
[[2 0]
 [0 2]]

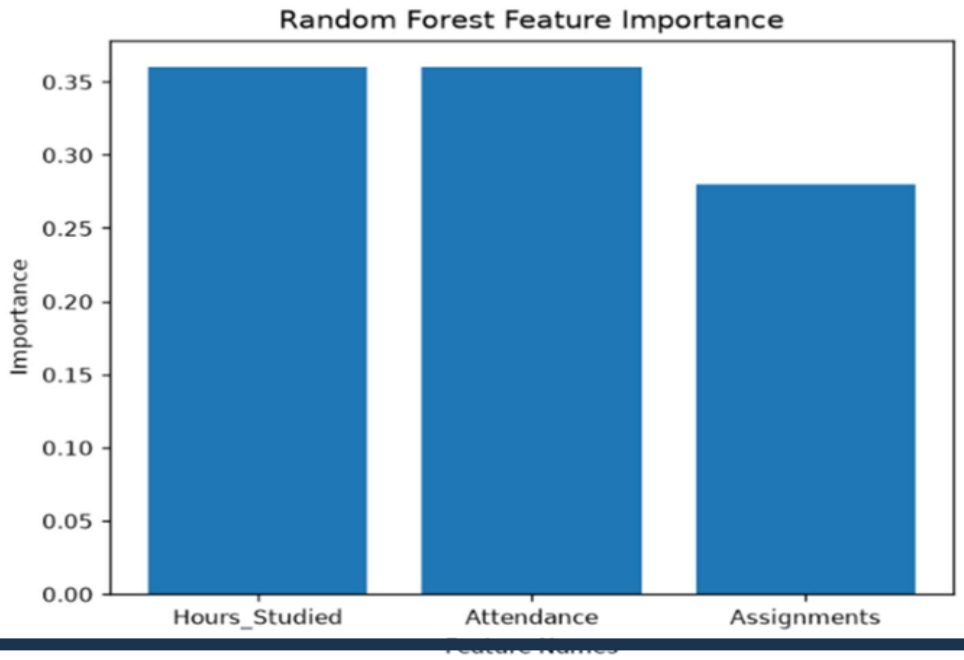
Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2
        Pass       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4


Process finished with exit code 0
```

Graph :



Sri Chandrasekharendra Saraswathi Viswa Mahavidyalaya (SCSVMV)
Deemed to be University | Accredited by NAAC with "A" Grade
(Sponsored and run by Sri Kanchi Kamakoti Peetam Charitable Trust)

Fundamentals Of Machine Learning Lab

Exp No. 8 : Implement Boosting Technique (AdaBoost) and Analyze Performance

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import matplotlib.pyplot as plt

# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
 *arrays: X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
n_estimators=50,
random_state=42
)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)
```

```
# Evaluate performance
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Feature Importance Graph
importance = model.feature_importances_

plt.bar(X.columns, importance)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.show()
```

Fundamentals Of Machine Learning Lab

Output :

```
Accuracy: 1.0

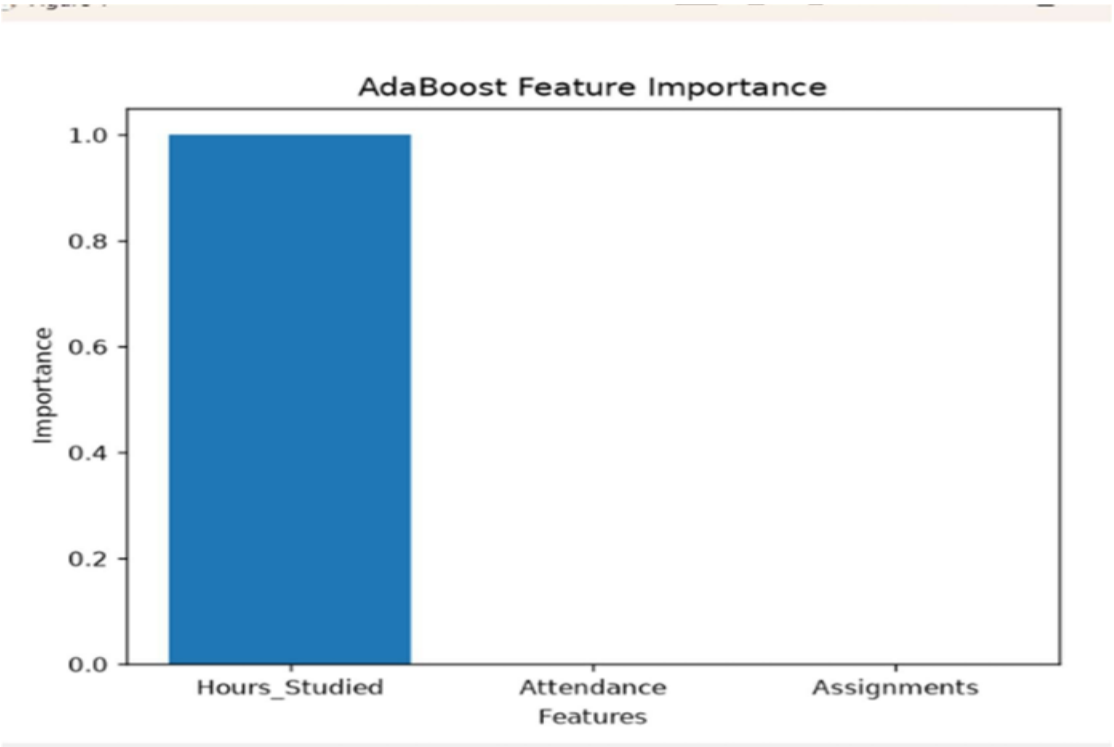
Confusion Matrix:
[[2 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2
        Pass       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4
```

Graph :



Fundamentals Of Machine Learning Lab

Exp No.9 : Implement a Basic Reinforcement Learning Model (Q-Learning)

Program:

```
#expo 9:implement a basic reinforcement learning model (Q-Learning)
import numpy as np
# Rewards matrix
R = np.array([
    [-1, -1, -1, -1,  0, -1],
    [-1, -1, -1,  0, -1,100],
    [-1, -1, -1,  0, -1, -1],
    [-1,  0,  0, -1,  0, -1],
    [ 0, -1, -1,  0, -1,100],
    [-1,  0, -1, -1,  0,100]
])
# Initialize Q-table
Q = np.zeros_like(R)
gamma = 0.8
episodes = 1000
# Training
for _ in range(episodes):
    state = np.random.randint( low: 0,  high: 6)

    while state != 5:
        actions = np.where(R[state] >= 0)[0]

        action = np.random.choice(actions)

        next_state = action

        Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])

        state = next_state

# Display Q-table
print("Q-Table:\n")
```

Fundamentals Of Machine Learning Lab

```
# Display Q-table
print("Q-Table:\n")
print(Q)
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("data.csv")

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable
y = data['Result']

# Convert labels to numeric
y = y.map({'Fail':0, 'Pass':1})
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
 *arrays: X, y, test_size=0.2, random_state=42
)
# Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# Build Neural Network
model = tf.keras.Sequential([
tf.keras.layers.Dense( units: 8, activation='relu', input_shape=(3,)),
tf.keras.layers.Dense( units: 4, activation='relu'),
```

Fundamentals Of Machine Learning Lab

```
model = tf.keras.Sequential([
tf.keras.layers.Dense( units: 8, activation='relu', input_shape=(3,)),
tf.keras.layers.Dense( units: 4, activation='relu'),
tf.keras.layers.Dense( units: 1, activation='sigmoid')
])
# Compile model
model.compile(
optimizer='adam',
loss='binary_crossentropy',
metrics=['accuracy']
)
# Train model
history = model.fit(
X_train,
y_train,
epochs=50,
validation_split=0.2,
verbose=1
)
# Predict
y_pred = model.predict(X_test)
# Convert probabilities to class labels
y_pred = (y_pred > 0.5).astype(int)
#Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

Output :

```
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\laksh\PycharmProjects\PythonProject1\main.py
Q-Table:

[[ 0  0  0  0 80  0]
 [ 0  0  0 64  0 100]
 [ 0  0  0 64  0  0]
 [ 0 80 51  0 80  0]
 [64  0  0 64  0 100]
 [ 0  0  0  0  0  0]]
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792597424.339298   25304 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from dif
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1792597429.323294   25304 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from dif
C:\Users\laksh\PycharmProjects\PythonProject1\.venv\Lib\site-packages\keras\src\layers\core\dense.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
I0000 00:00:1792597432.444622   25304 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or the
Epoch 1/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step - accuracy: 0.5000 - loss: 0.6836 - val_accuracy: 1.0000 - val_loss: 0.6822
Epoch 2/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 110ms/step - accuracy: 0.9167 - loss: 0.6823 - val_accuracy: 1.0000 - val_loss: 0.6813
Epoch 3/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 106ms/step - accuracy: 0.9167 - loss: 0.6811 - val_accuracy: 1.0000 - val_loss: 0.6804
Epoch 4/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 103ms/step - accuracy: 0.9167 - loss: 0.6797 - val_accuracy: 1.0000 - val_loss: 0.6794
Epoch 5/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 107ms/step - accuracy: 0.9167 - loss: 0.6783 - val_accuracy: 1.0000 - val_loss: 0.6785
Epoch 6/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 103ms/step - accuracy: 0.9167 - loss: 0.6769 - val_accuracy: 1.0000 - val_loss: 0.6776
Epoch 7/50
```